

Name: Teras Abebe

CMSC471_1445_FA2026

6. Written analysis [40 points]

Submit your answers in a PDF named pa01.pdf.

Support your conclusions using your implementation, formal reasoning, and experimental results. A returned path may differ from the sample output while still being correct if it has the same optimal cost.

Question 1: Heuristic design and admissibility [15 points]

For each of the three cost measures—**steps**, **Scrabble**, and **frequency**—describe the heuristic implemented in your program.

For each heuristic:

1. Give a precise formula or pseudocode.

Conventions used in Q1:

State1 (current state before action to next state)

State2 (new state that was entered)

X is placeholder for Letter being added

- **Steps:**
 - add 1 to the current total cost (the current total that was needed to get to state2)
 - If cost metric is steps:
 - then add +1 to the current total
 - then we return the new total
 - Else if cost metric is **Scrabble**
 - add the value of the letter being added following the letter value table found in the Pa instructions pdf.
 - if cost metric is scrabble:
 - then add letterValue(X) to current total
 - return new total
 - Lastly if **Frequency**
 - Add 1 plus the rarity value of the new word that's assigned to the word inside our supplied dictionary using the word as the key.
 - if cost metric frequency:
 - then add 1 + dictionary(newWord) to current total
 - return new total
2. Explain what unavoidable remaining costs of lower-bounds.
 - Steps
 - Add 1 for every letter that isn't correct and isn't in the correct position
 - Scrabble
 - The unavoidable cost here is the letters in goal state that aren't in my current state. Eventually we will have to pay them. Current state -> goal state
 - Frequency
 - Adding one for every letter that isn't correct plus the rarity value of the goal word.

3. Explain why it cannot overestimate the true optimal remaining cost.

We can not overestimate the true optimal remaining cost because the goal state must take as many letters(in the correct position) that are missing from our current state plus the words that are used to get to the goal state must be legal.

So for ask to why

We know at least 3 changes need to be made but since for example 'say' isn't a word, we need to account for the words that are usable. So:

ask->ass->ahs->aha-> wha->why

So 3 letters needed to be replaced and 5 needed stages to actually get to the final stage.

Your explanation must be specific to the corresponding cost measure. Merely stating that the heuristic "is a lower bound" is insufficient.

Question 2: Does the heuristic actually help? [10 points]

Word pair	cost	Ucs cost	Ucs nodes	A* cost	A* nodes	reduction
Why ask	steps	5	773	5	16	97.9
Why ask	scrabble	9	765	9	13	98
Why ask	frequency	38.8	808	38.8	634	21.5
Bear duck	Steps	4	691	4	6	99
Bear duck	scrabble	11	2981	11	6	99.8
Bear duck	frequency	38.2	1084	38.2	302	72

Then answer:

1. Did UCS and A* always return the same optimal cost? If not, identify and investigate the discrepancy.

Yes it actually gave me the equal for all of my cases.

2. For each word pair, under which cost measure did A* produce the largest percentage reduction in expanded nodes?

Why ask with scrabble gave me a 98%

And Bear to duck with scrabble gives me a 99%!

5

3. For each word pair, under which cost measure was the heuristic least informative?

For both pairs frequency gave me the least information, being as they both gave the least amount of reduction. (21% and 71%)

4. Explain the differences using the definitions of your heuristics, not runtime alone.

Steps and scrabble reduced the number of nodes drastically compared to frequency. Steps and scrabble gave enough information about how many letters needed to be changed and cost of those letters when using scrabble. While frequency was the less of the 3, it did not reduce the amount at all and led to more nodes being expanded.

Different optimal paths are acceptable when their total costs are equal.

Question 3: Reversibility and direction-dependent costs [5 points]

DOGCAT actions are reversible: if one legal word can be changed into another, the reverse change is also legal.

For each cost measure—**steps**, **Scrabble**, and **frequency**—determine whether the optimal cost from w_1 to w_2 must always equal the optimal cost from w_2 to w_1 .

For each cost measure:

1. State **yes** or **no**.
2. Justify your conclusion.
3. If the answer is no, give a concrete counterexample using words from the

provided dictionary. Clearly distinguish between the reversibility of an action and the symmetry of its cost.

- Steps
 - Yes, every action costs exactly 1, going in reverse optimally has to have the exact same amount of stages.
- Scrabble
 - No, for this situation not all words have the same value. Each letter has different values so each stage would grant the problem a different value. Which is why going in reverse would grant a different cost. So Even though we could reverse the action, the cost does not need to be the same
 - Example: 'why to ask' when taking steps on 'why' to the next stage 'wha', a cost 1 while reverting back to y would cost 4.
- Frequency
 - No, cost depends on the new words frequency, which isn't consistent with every word. The frequencies are unique to each word so although we can revert the action back and forth from 1 word to another, that

process might not necessarily give you the same cost.

- Example: cat $\rightarrow$ sat would give a cost of $1 + \text{rarity}(\text{sat})$ and doing a reverse would give $1 + \text{rarity}(\text{cat})$ so that shows even though we could reverse, we would get a different value.

Question 4: LLM heuristic audit [10 points]

Give a generative AI system the following prompt:

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary.

The baseline heuristic $h_0(w)$ is the number of positions in which state w differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal.

By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

Submit:

1. The model used, the exact prompt, and its proposed heuristic. A complete conversation transcript is not required.
- Gemini flash
 - “In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary. The baseline heuristic $h_0(w)$ is the number of positions in which state w differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal. By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost. “

$$h_{R^+}(n) = \sum_{i \in D(n,g)} (1 + r_i^{\min})$$

where:

- $D(n, g)$ is the set of index positions where current state n differs from target goal g .
- $r_i^{\min} = \min\{\text{rarity}(w) \mid w \in V \text{ and } w[i] = g_i\}$ is the minimum rarity among all dictionary words having goal letter g_i at position i .

Why this is the optimal choice:

1. **Admissible:** To solve each mismatch position i , any valid path must eventually transition into a word containing g_i at that index. The cost of entering that word is at least $1 + r_i^{\min}$.
2. **More Informative:** Because $r_i^{\min} \geq 0$, $h_{R^+}(n) \geq h_0(n)$ for every state, and $h_{R^+}(n) > h_0(n)$ whenever any target letter g_i carries a positive minimum rarity in the dictionary.

$$h_{R^+}(n) = \sum_{i=1}^L \mathbb{I}(n[i] \neq g[i]) \cdot (1 + r_i^{\min})$$

2. A precise formula or pseudocode for the proposed heuristic.

"# HEURISTIC EVALUATION (Run at each expanded node)

def heuristic_h_R_plus(current_word, goal_word, R_min):

"""

Calculates the rarity-aware admissible heuristic $h_{R^+}(n)$.

Parameters:

current_word (str): Current word state n

goal_word (str): Target goal word g

R_min (list[float]): Precomputed minimum rarities for target letters

Returns:

float: Estimated remaining path cost

"""

if current_word == goal_word:

return 0.0

h_val = 0.0

for i in range(len(current_word)):

if current_word[i] != goal_word[i]:

h_val += 1.0 + R_min[i]

return h_val
“

3. Determine whether the heuristic is admissible. Give either a general proof or a concrete counterexample.

It's admissible because each move changes 1 letter and $h(n)$ moves would be needed to eventually reach the goal. Each move cost being at least $1 + R$ minimum, so remaining cost being $h(n) * (1 + R_{\text{minimum}})$

4. If the proposal is invalid, make the smallest repair you can or replace it with a safe alternative. Might be safer if it used just a global minimum instead of considering a position specific sum. Using $h(n)(1 + R_{\text{minimum}})$

5. Test the verified or repaired heuristic against h_0 on the two frequency-cost problems from Question 2. Compare

solution costs and numbers of expanded nodes, then conclude whether the evidence supports using the proposed heuristic.

To do..

Experimental tests can reveal counterexamples, but passing a collection of tests does not by itself prov